

MEDISPHERE

Care & Medication Safety Platform

Milestone 4 Documentation — Team C

1. Project Overview & Objectives

1.1 Problem Statement

Healthcare decisions involve multiple layers of safety checks, including medication combinations, dosage limits, adherence to clinical guidelines, and patient-specific conditions. When these checks are performed manually, the result is often inconsistent validation, missed safety conditions, difficulty maintaining and updating rules, repeated verification effort across teams, and a lack of centralized information.

MediSphere's Milestone 4 addresses this gap by introducing three independent, automatable safety-checking services that a clinician-facing frontend can call in a consistent, structured way.

1.2 Project Objectives

MediSphere provides a centralized interface backed by independent backend services. The Milestone 4 objectives are to:

- Validate patient data against clinical guidelines
- Check the safety of medications entered as part of a care plan
- Detect interactions between combinations of drugs
- Persist the results of each safety check
- Provide structured, predictable API responses
- Maintain each safety concern as a separately deployable module
- Provide a single, common frontend across all three modules

1.3 Team & Technology Stack

Team Members	Technology Stack
Shashikant, Saranya, Nhowmitha, Pinki, Raja, Somesh, Nilesh, Karishma	Frontend: HTML, CSS, JavaScript Backend: Java, Spring Boot Database: PostgreSQL Testing: JUnit, Spring Boot Test Communication: REST API, HTTP, JSON

1.4 Scope of Milestone 4

Milestone 4 focuses specifically on the three safety-validation services and the frontend surface that exposes them. It does not include authentication/authorization, patient-record management, or clinical-decision-support beyond the documented rule sets — those are treated as out of scope for this milestone and are candidates for later iterations.

1.5 Key Terminology

Term	Definition
Care Plan	A structured set of medications, monitoring instructions, and clinical parameters associated with a patient.

Guideline	A named set of clinical rules (e.g. threshold and monitoring rules) that a care plan is evaluated against.
Violation	A rule outcome that causes the overall result to fail (NON_COMPLIANT / UNSAFE).
Warning	A rule outcome that is flagged for attention but does not by itself fail the overall result.
Medicine Pair	One of the $n(n - 1) / 2$ unique two-medicine combinations generated from a submitted medicine list.

2. Clinical Guideline Compliance

2.1 Overview

The Clinical Guideline Compliance component validates patient and care-plan information against configured clinical guidelines and active clinical rules. Its primary purpose is to determine whether the information associated with a patient's care plan satisfies the applicable clinical requirements.

Manual verification of clinical conditions can result in inconsistent decisions, missed safety conditions, repeated verification, and difficulty maintaining clinical rules. This module addresses that problem with a centralized, structured validation process: it receives patient and care-plan information, identifies the applicable active guideline, loads the associated rules, evaluates the patient's information against those rules, checks medication and allergy conflicts, and generates a compliance result.

Output is categorized as COMPLIANT or NON_COMPLIANT, with additional guideline code, violations, warnings, and timestamp information.

2.2 Purpose

- Validate patient information against configured clinical guidelines
- Identify the active guideline applicable to the patient's condition
- Load the rules associated with the active guideline
- Evaluate clinical thresholds
- Verify medication and allergy conflicts
- Identify violations and warnings
- Generate a structured compliance result
- Provide the result to the frontend through an API
- Maintain consistency in clinical-rule evaluation

2.3 Input Data

Input	Description
Patient ID	Identifies the patient being evaluated
Care Plan ID	Identifies the care plan being validated
Medical Condition	Represents the patient's relevant clinical condition
Age	Patient age used during guideline evaluation
HbA1c Value	Laboratory measurement used for applicable guideline rules

HbA1c Monitoring Interval	Indicates the monitoring frequency
Allergies	Patient allergy information
Current Medications	Medications currently being taken
Care-plan Medications	Medications specified in the care plan

2.4 Guideline Identification & Rule Loading

The module first determines the applicable guideline, then loads its associated rules, rather than evaluating isolated values or hard-coding unrelated checks:

Patient Information → Identify Active Guideline → Load Active Rules → Evaluate Clinical Conditions

The PPT identifies three implemented rule categories:

- HbA1c maximum threshold
- HbA1c monitoring interval
- Medication/allergy review

2.5 HbA1c Maximum Threshold Validation

The patient's HbA1c value is evaluated against a configured maximum threshold:

HbA1c Value → Load Guideline Threshold → Compare Patient Value → Determine Compliance

If the patient's value violates the configured threshold, the module records the corresponding violation. The exact numerical threshold is not specified in the PPT, so it should be treated as a configured clinical rule rather than a fixed value.

2.6 HbA1c Monitoring Interval Validation

This rule evaluates whether the monitoring frequency associated with the patient or care plan satisfies the requirements of the active guideline, so compliance considers planned monitoring, not just the current lab value:

Patient Monitoring Interval → Compare with Active Guideline Rule → Identify Violation if Requirement Is Not Met → Add Violation/Warning

2.7 Medication and Allergy Review

The system evaluates patient allergies, current medications, and care-plan medications to identify potential conflicts between care-plan medications and known allergies:

Patient Allergies + Current Medications + Care-plan Medications → Medication/Allergy Review → Identify Conflicts → Generate Violation or Warning

2.8 Compliance Evaluation Process

The complete workflow consists of six major stages: Patient Information → Identify Active Guideline → Load Active Rules → Evaluate Thresholds → Check Medication/Allergy Conflicts → Generate Compliance Result.

2.9 Compliance Result

A COMPLIANT result indicates the evaluated information satisfies the applicable rules; a NON_COMPLIANT result indicates one or more applicable rules have been violated. The response also provides guideline code, violations, warnings, and timestamp, giving the user more context than a simple Boolean.

Violations represent conditions that cause the submitted information to fail an applicable rule, while warnings provide additional information that may require attention without representing a direct compliance failure.

2.10 Database Integration

The module uses module-specific tables within the shared PostgreSQL database: `clinical_guideline`, `guideline_rule`, and `compliance_result`.

2.11 Technical Architecture & Health Monitoring

Implemented as an independent Spring Boot service (01-Clinical-Guideline-Compliance, Spring Boot Service :8102) using Java, Spring Boot, Spring Web MVC, Spring JDBC, and Bean Validation. The service exposes a health endpoint at `/api/clinical-guidelines/health`, which the frontend uses to display UP or DOWN status.

The service follows a layered structure typical of a Spring Boot application: a Controller layer that accepts the incoming JSON request and performs basic request shape validation (via Bean Validation annotations), a Service layer that contains the guideline-identification and rule-evaluation logic described above, and a Repository/JDBC layer responsible for reading guideline and rule definitions from PostgreSQL and persisting the generated `compliance_result` row.

2.12 Detailed Rule Logic

HbA1c Maximum Threshold — Worked Example

Conceptually, the rule loads the maximum permissible HbA1c value configured for the patient's active guideline and compares it against the HbA1c value submitted in the request. If the submitted value is greater than the configured maximum, a violation is recorded (for example, an entry such as "HbA1c value exceeds maximum threshold for guideline"). If the value is within range, no violation is added for this rule and evaluation continues to the next configured rule.

HbA1c Monitoring Interval — Worked Example

This rule compares the monitoring interval submitted with the patient/care-plan information against the interval required by the active guideline. Because monitoring frequency is a planning concern rather than a single measured value, a mismatch is treated as either a violation or a warning depending on how the guideline rule is configured — for example, a monitoring interval that is less frequent than required could be modeled as a violation, while a borderline interval could be modeled as a warning that a clinician should review.

Medication/Allergy Review — Worked Example

For each medication listed in current medications and care-plan medications, the module checks whether that medication (or a known cross-reactive substance) appears in the patient's allergy list. A positive match is recorded as a conflict; conflicts are surfaced as violations because they represent a direct patient-safety risk, distinct from the softer, monitoring-related warnings described above.

2.13 Error Handling & Edge Cases

- Missing required fields (e.g. no Patient ID or Care Plan ID) are rejected at the request-validation stage before guideline evaluation begins.
- A patient/condition with no matching active guideline results in a clearly identified "no applicable guideline" outcome rather than a silent pass.
- An empty allergies, current-medications, or care-plan-medications list is treated as "nothing to check" for that rule rather than as an error.
- Multiple violations across different rules are all collected and returned together, rather than the evaluation stopping at the first violation found.

2.14 Technical Significance

This component provides a centralized mechanism for evaluating clinical information against configured rules, reducing dependence on manual verification. It is independently deployable and maintains its own business logic and database tables, allowing it to evolve independently from the Careplan Safety and Drug Interaction modules.

3. Careplan Safety Checks

3.1 Overview

The Careplan Safety Validation component validates medication information entered as part of a patient's care plan — medication names, dosage, frequency, and duration — applying predefined safety rules before the care-plan medication information is considered safe.

The module validates the request, evaluates configured safety rules, collects violations, produces a SAFE or UNSAFE result, and stores the result.

3.2 Purpose

- Validate care-plan medication input, dosage, frequency, treatment duration, and medication names
- Identify safety violations
- Generate a SAFE or UNSAFE result
- Store the safety-check result
- Provide patient history and stored-result statistics

3.3 Input Parameters

Input	Description
Patient ID	Identifies the patient
Medication	Medication included in the care plan
Dosage	Quantity or dosage specified
Unit	Unit associated with the dosage
Frequency	Medication schedule
Duration	Duration for which the medication is prescribed

3.4 Safety Rules

Before rule evaluation, the submitted information passes through request validation: Careplan Input → Request Validation → Safety Rule Evaluation. Four safety rules are then applied:

- Dosage Limit — submitted dosage compared against a configured limit; if exceeded, generates `DOSAGE_LIMIT_EXCEEDED`
- Frequency Validation — invalid schedules generate `INVALID_FREQUENCY`
- Duration Validation — durations greater than 365 days generate `INVALID_DURATION`
- Medication Validation — invalid or short medication names generate `INVALID_MEDICATION`

3.5 Safety Rule Evaluation & Violation Collection

Careplan Input → Request Validation → Dosage / Frequency / Duration / Medication Validation → Collect Violations → SAFE / UNSAFE → Store Result

Rather than stopping at the first problem, the module collects all safety violations identified during evaluation, so a single request can surface more than one invalid condition.

3.6 Final Safety Result

A SAFE result indicates the submitted care-plan information passed the configured safety checks; an UNSAFE result indicates one or more safety violations were detected, with individual violation information providing additional context.

3.7 API Endpoint

POST /api/careplans/safety-check

Frontend → HTTP POST → Careplan Service :8100 → Controller → Safety Evaluation → PostgreSQL → Structured Response

3.8 Database Integration & Patient History

The module uses a dedicated table, `careplan_safety_result`, within the shared PostgreSQL database. The system also provides patient history and stored-result statistics, allowing previously processed safety checks to be reviewed rather than treating every request as an isolated operation.

3.9 Service Health Monitoring

The service exposes a health endpoint at `/api/careplans/health`, which the frontend displays as UP or DOWN.

3.10 Detailed Rule Logic

Dosage Limit — Worked Example

The submitted dosage and unit are compared against a configured maximum for that medication. If the submitted dosage exceeds the configured limit, the service adds a `DOSAGE_LIMIT_EXCEEDED` violation that identifies the medication, the submitted dosage, and the configured limit, so the caller can see exactly why the plan failed.

Frequency Validation — Worked Example

The submitted frequency is checked against the set of schedules the system recognizes as valid (for example, expected values describing how many times per day or per week a medication should be taken). A frequency that does not match a recognized, well-formed schedule produces an `INVALID_FREQUENCY` violation.

Duration Validation — Worked Example

The submitted duration, expressed in days, is compared against the configured maximum of 365 days. A duration of 366 days or more produces an `INVALID_DURATION` violation; a duration within the limit passes this rule without further action.

Medication Validation — Worked Example

The submitted medication name is checked for basic input quality — for example, a name that is blank, null, or below a minimum length threshold is treated as invalid and produces an `INVALID_MEDICATION` violation. This check runs independently of the dosage, frequency, and duration checks, so a care plan can fail on medication-name quality even if its numeric fields are otherwise valid.

3.11 Error Handling & Edge Cases

- A request missing Patient ID or Medication fails request validation before any safety rule is evaluated.
- A care plan can trigger more than one violation simultaneously (for example `DOSAGE_LIMIT_EXCEEDED` and `INVALID_DURATION` together); all applicable violations are collected rather than only the first one found.
- A zero or negative dosage/duration is treated as invalid input at the request-validation stage rather than being evaluated against the dosage or duration rules.
- Results are persisted even when the outcome is UNSAFE, so that unsafe submissions remain visible in patient history rather than being discarded.

3.12 Technical Significance

By separating dosage, frequency, duration, and medication validation into explicit safety rules, the system provides a structured method for detecting unsafe care-plan configurations, while maintaining independent business logic and its own database table.

4. Drug Interaction Validation

4.1 Overview

The Drug Interaction Validation component checks combinations of medicines against configured drug-interaction rules. It systematically evaluates submitted medicines, identifies relevant medicine pairs, searches the interaction database, and returns matching interactions, including severity, description, and recommendation.

4.2 Purpose

- Accept a list of medicines and clean/normalize the submitted names
- Remove duplicate medicines and verify that medicines exist
- Generate unique medicine pairs
- Search the interaction database and check interactions in both drug orders
- Identify matching interaction rules and return structured results

4.3 Input Format

```
{
  "medicines": ["Aspirin", "Warfarin", "Ibuprofen"]
}
```

4.4 Input Cleaning & Medicine Existence Validation

The module removes null values, removes blank values, trims medicine names, and removes duplicates, before validating that the submitted medicines exist in the configured medicine data:

Submitted Medicines → *Clean Input* → *Validate Medicine Existence* → *Generate Valid Medicine Set*

4.5 Unique Medicine Pair Generation

The number of unique pairs is calculated as $n(n - 1) / 2$, where n is the number of medicines. For three medicines (Aspirin, Warfarin, Ibuprofen), this produces three pairs: Aspirin+Warfarin, Aspirin+Ibuprofen, and Warfarin+Ibuprofen.

4.6 Interaction Database Search

The service searches two tables within the shared PostgreSQL database — medicines and drug_interactions — for matching rules, and checks both orders of a medicine pair (bidirectional checking), so a pair submitted in reverse order still matches a stored interaction record:

Medicine Pair → *Interaction Database* → *Search for Matching Rule* → *Interaction Found / No Matching Interaction*

4.7 Example Interaction Result

Field	Value
Drug 1	Aspirin
Drug 2	Warfarin

Severity	HIGH
Description	Potentially increased bleeding risk
Recommendation	Consult a healthcare professional

The exact severity categories beyond the demonstrated HIGH value are not specified in the PPT, so additional categories should not be assumed as part of the current documented implementation.

4.8 API Endpoint

POST /api/interactions/check

The service runs on port :8101, identified in the project structure as 03-Drug-Interaction-Validation.

4.9 Database Integration & Health Monitoring

The medicines table provides available medicine information, while drug_interactions contains the configured interaction rules. The service exposes a health endpoint at /api/interactions/health, displayed by the frontend as UP or DOWN.

4.10 Why Unique Pairs and Bidirectional Checking Matter

Generating unique pairs rather than checking every ordered combination avoids redundant work: for n medicines there are n^2 ordered combinations but only $n(n - 1) / 2$ unordered pairs, so pair generation roughly halves the number of lookups compared to a naive ordered-pair approach. Because an interaction rule could be stored as (Aspirin, Warfarin) or (Warfarin, Aspirin) depending on how it was entered, the search still checks both orders for each generated pair — so the halving comes from not re-checking the same unordered pair twice, not from skipping either order of a given pair.

4.11 Error Handling & Edge Cases

- A single medicine ($n = 1$) produces zero pairs, so the module returns an empty interaction result rather than an error.
- An empty or entirely blank/null medicine list is rejected at input cleaning before pair generation is attempted.
- A medicine name that does not exist in the medicines table is excluded from pair generation, and the response can indicate which submitted names were not recognized.
- When no configured interaction rule matches a given pair, that pair is simply reported as having no interaction rather than being treated as an error condition.
- Duplicate medicine names in the input (including differing only by case or whitespace) are normalized and de-duplicated before pairs are generated, so the same pair is never evaluated twice.

4.12 Technical Significance

Unique pair generation ensures every combination is evaluated efficiently, and database-driven interaction rules allow interaction information to be maintained separately from application logic. The service is independently deployable and communicates with the common frontend through a REST API.

5. Common Microservice Architecture

5.1 Overview

MediSphere uses a microservice-based modular architecture consisting of three independent Spring Boot backend services and a common frontend:

- Clinical Guideline Compliance — port 8102
- Careplan Safety Checks — port 8100
- Drug Interaction Validation — port 8101

This allows each safety module to maintain independent business logic while sharing a common frontend and PostgreSQL database.

5.2 Architectural Components

- Independent backend services
- Common frontend
- Shared PostgreSQL database
- Module-specific database tables
- REST-based communication
- Independent business logic

5.3 Project Structure

```
MEDISPHERE
├── 01-Clinical-Guideline-Compliance (Spring Boot Service :8102)
├── 02-Careplan-Safety-Checks (Spring Boot Service :8100)
├── 03-Drug-Interaction-Validation (Spring Boot Service :8101)
├── 04-Frontend (Common User Interface)
└── POSTMAN_COLLECTION.json
```

5.4 Shared PostgreSQL Database

Module	Database Tables
Clinical Guideline	clinical_guideline, guideline_rule, compliance_result
Careplan	careplan_safety_result
Drug Interaction	medicines, drug_interactions

5.5 Technology Stack

Frontend: HTML, CSS, JavaScript, custom responsive UI, JSON-based API communication. Backend: Java, Spring Boot, Spring Web MVC, Spring JDBC, Bean Validation. Database & Testing: PostgreSQL with SQL schema initialization and module-specific tables, Maven, JUnit, Spring Boot Test, and a Postman API collection.

5.6 Layered Structure Within Each Service

Although each backend service is independently deployable, all three follow the same internal layering for consistency:

- Controller layer — accepts the HTTP POST request, performs request-shape validation with Bean Validation, and maps the JSON payload to a request object.
- Service layer — contains the module's business/safety rules and orchestrates the evaluation sequence described in each module's section above.
- Repository/JDBC layer — reads configuration (guidelines, rules, dosage limits, interaction records) from PostgreSQL and persists generated results.
- Response layer — assembles the structured JSON response (result, violations, warnings, and any module-specific fields such as severity or guideline code).

5.7 Non-Functional Considerations

Scalability

Because each service is independently deployable, individual modules can be scaled horizontally based on their own load — for example, Drug Interaction Validation could be scaled independently of Clinical Guideline Compliance if it receives a disproportionate share of traffic.

Observability

Each service's dedicated health endpoint (documented in its own section above) gives the frontend a simple per-module UP/DOWN signal, which can also be used by external monitoring or orchestration tooling.

Data Consistency

All three services share one PostgreSQL instance, so referential and transactional consistency within a single request is straightforward, while module-specific tables keep each service's schema changes from affecting the others.

Security Considerations (Out of Scope for Milestone 4)

Authentication, authorization, and audit logging of who performed a given safety check are not part of the Milestone 4 scope as documented, but would be natural extensions given that the platform handles patient-identifying and clinical information.

6. Data Model Overview

The six module-specific tables introduced across the three services can be grouped by the module that owns them, as summarized below. Column-level schema is not specified in the PPT, so the table below describes each table's role rather than its exact DDL.

6.1 Clinical Guideline Compliance Tables

Table	Role
clinical_guideline	Stores the definition of each named clinical guideline (e.g. the guideline code referenced in compliance results).
guideline_rule	Stores the individual rules belonging to a guideline — HbA1c maximum threshold, HbA1c monitoring interval, and medication/allergy review configuration.
compliance_result	Stores the generated COMPLIANT / NON_COMPLIANT outcome for a given patient and care plan, along with violations, warnings, and timestamp.

6.2 Careplan Safety Tables

Table	Role
careplan_safety_result	Stores the generated SAFE / UNSAFE outcome for a submitted care-plan medication, along with any collected violations, supporting the patient-history and statistics features.

6.3 Drug Interaction Tables

Table	Role
medicines	Stores the set of medicine names recognized by the system, used during medicine-existence validation.

drug_interactions	Stores configured interaction rules between medicine pairs, including severity, description, and recommendation.
-------------------	--

Note: Exact column names and types are not specified in the source PPT; the descriptions above reflect the role each table plays in the documented workflow rather than an asserted schema.

7. End-to-End Application Flow

7.1 General Processing Model

From user input to final result display, the application routes each request through a clear service pipeline, validates safety rules, queries the shared database, and returns a structured response the frontend can render immediately:

User Input → Frontend → JSON Request → Appropriate Spring Boot Service → Request Validation → Business/Safety Rule Evaluation → PostgreSQL Query → Structured Response → Frontend → Result Display

Request/Response format: every service accepts a JSON request payload and returns a structured JSON response.

Error handling: invalid input, missing medicines, and safety violations are all identified and communicated before results are returned to the caller. Response time expectations: the flow is designed for fast, near-immediate feedback in the UI, since each request only touches its own module's tables.

7.2 Clinical Guideline Flow

User Input → Clinical Guideline Service :8102 → Validation → Database Query → Guideline Evaluation → Compliance Result → Frontend

7.3 Careplan Safety Flow

User Input → Careplan Service :8100 → Request Validation → Safety Rule Evaluation → Database Query / Result Storage → SAFE / UNSAFE → Frontend

7.4 Drug Interaction Flow

User Input → Drug Interaction Service :8101 → Medicine Validation → Pair Generation → PostgreSQL Interaction Query → Interaction Response → Frontend

8. Detailed Example: Drug Interaction

The user enters Aspirin + Warfarin. The complete processing sequence is:

- User enters medicines: Aspirin + Warfarin
- Frontend creates a JSON request
- Request is sent to POST /api/interactions/check
- DrugInteractionController receives the request
- DrugInteractionService processes the submitted medicines
- Medicines are validated and cleaned
- Unique medicine pairs are generated
- PostgreSQL is queried
- Matching interactions are identified
- InteractionResponse is created
- JSON response is returned
- Frontend displays the interaction result and severity (HIGH)

The result: Drug 1 — Aspirin, Drug 2 — Warfarin, Severity — HIGH, Description — potentially increased bleeding risk, Recommendation — consult a healthcare professional. The user receives a structured safety result without directly interacting with the PostgreSQL database.

8.1 Walking Through the Same Example With a Third Medicine

If the user instead submits Aspirin, Warfarin, and Ibuprofen, the same flow applies but with three generated pairs instead of one: Aspirin+Warfarin, Aspirin+Ibuprofen, and Warfarin+Ibuprofen. Each pair is looked up independently (in both orders), so the final response can contain zero, one, or multiple interaction entries depending on how many of the three pairs have a configured interaction rule. This illustrates why the module collects and returns a list of interaction results rather than a single pass/fail flag — a three-medicine care plan can be partially safe and partially flagged at the same time.

9. Technical Benefits of the Architecture

9.1 Independent Service Processing

Each module owns its own validation and business logic — Clinical Guideline Compliance handles clinical rules, Careplan Safety handles care-plan medication safety, and Drug Interaction Validation handles medication combinations — preventing unrelated business rules from being mixed together.

9.2 Shared Database Efficiency

The services use a common PostgreSQL instance while maintaining separate tables for each module, reusing common database infrastructure while keeping module data organized.

9.3 Modular Development

Independent services allow development teams to build and extend individual modules without modifying the entire application — for example, Drug Interaction rules can change without affecting the Clinical Guideline service.

9.4 Testing and Debugging

The project uses JUnit, Spring Boot Test, and Postman API collections for testing. Because each service has a clear request path, problems can be isolated more easily during testing and debugging.

9.5 Structured API Communication

The system uses JSON request and response formats, creating a predictable communication mechanism between frontend and backend. The frontend never communicates directly with PostgreSQL — requests always pass through the appropriate Spring Boot service, where validation and business logic are performed before the result is returned.

9.6 Testing Strategy

Each service is covered using JUnit and Spring Boot Test at the unit and integration level, with a Postman collection (POSTMAN_COLLECTION.json) providing end-to-end, black-box verification of every documented endpoint. This combination allows individual rule logic (e.g. the dosage-limit or HbA1c-threshold comparisons) to be unit-tested in isolation, while the Postman collection exercises the full HTTP → service → PostgreSQL → response path for each module.

- Unit tests around each safety rule (dosage, frequency, duration, medication; HbA1c threshold and monitoring interval; medicine-pair generation) verify rule logic independently of the web layer.
- Spring Boot Test integration tests exercise the Controller → Service → Repository path against the shared PostgreSQL schema.

- The Postman collection provides a repeatable, documented set of example requests and expected responses for manual or CI-driven verification of all three health endpoints and all three primary endpoints.

10. Overall Technical Significance

The Milestone 4 implementation provides a modular healthcare safety platform focused on three major areas: clinical guideline compliance, care-plan medication safety, and drug interaction validation.

The Clinical Guideline Compliance module evaluates patient and care-plan information against configured clinical rules and produces COMPLIANT or NON_COMPLIANT results. The Careplan Safety module validates medication dosage, frequency, duration, and medication information and produces SAFE or UNSAFE results. The Drug Interaction module evaluates combinations of medicines by generating unique medicine pairs and comparing them against configured interaction rules.

Together, these modules provide a layered safety-validation system in which different healthcare safety concerns are handled by dedicated services. The overall architecture combines independent Spring Boot services, a common responsive frontend, REST/HTTP/JSON communication, and a shared PostgreSQL database with module-specific tables — providing clear separation of business responsibilities while maintaining a unified user experience.

10.1 Summary Comparison of the Three Modules

Module	Summary
Clinical Guideline Compliance	Port 8102 · Evaluates patient/care-plan data against guideline rules · Result: COMPLIANT / NON_COMPLIANT · Tables: clinical_guideline, guideline_rule, compliance_result
Careplan Safety Checks	Port 8100 · Evaluates dosage, frequency, duration, medication name · Result: SAFE / UNSAFE · Table: careplan_safety_result
Drug Interaction Validation	Port 8101 · Evaluates medicine pairs for known interactions · Result: list of interactions with severity · Tables: medicines, drug_interactions

11. Assumptions, Limitations & Future Work

11.1 Assumptions

- Numerical thresholds and limits (e.g. the HbA1c maximum threshold, medication dosage limits) are treated as configured, guideline- or medication-specific values rather than fixed constants, since exact figures are not specified in the source PPT.
- "Active" guidelines and rules are assumed to be selected based on the patient's medical condition, with inactive or superseded guidelines excluded from evaluation.
- Frequency schedules are assumed to be validated against a recognized, finite set of schedule formats rather than free-text input.

11.2 Limitations

- Exact column-level schema for the six database tables is not specified in the source PPT and has been described here at the role/purpose level only.
- Authentication, authorization, and audit logging are not covered in the documented Milestone 4 scope.
- Severity classification for drug interactions is only demonstrated for the HIGH value; the complete set of severity levels is not enumerated in the source material.

11.3 Suggested Future Work

- Introduce authentication/authorization so that safety checks are tied to an authenticated clinician or system identity.
- Expand the drug-interaction severity model beyond HIGH (e.g. MODERATE, LOW) with corresponding frontend styling.
- Add configurable, guideline-specific dosage limits per medication rather than a single global limit.
- Extend patient history/statistics reporting across all three modules, not only Careplan Safety.